Спринт 14, тема «Вёрстка кодом»

Яндекс Практикум

Привет! Мы сделали для вас памятку, в ней — краткое изложение того, что было в уроке, и ключевые фрагменты кода.

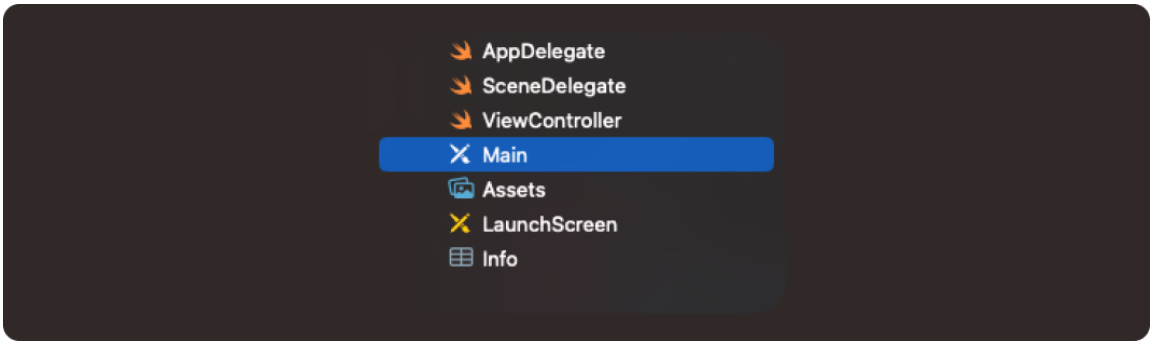
🚀 Скачайте документ, чтобы быстро возвращаться к нему при необходимости.

AutoLayout — мощный инструмент: с его помощью можно создавать адаптивный интерфейс в iOS. AutoLayout позволяет задавать правила, или констрейнты. А они определяют, как должны располагаться и изменяться размеры элементов интерфейса в зависимости от размера и ориентации экрана.

Удаление Storyboard

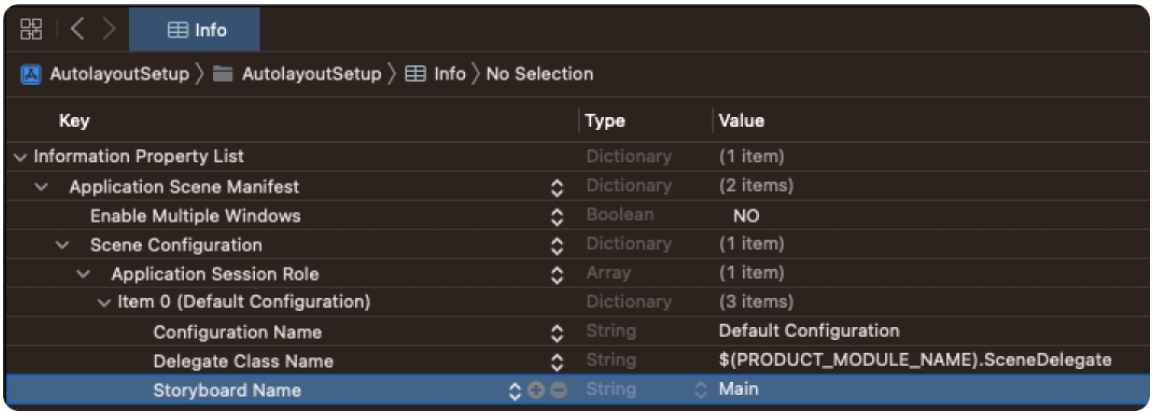
🔓 Чтобы отказаться от **Storyboard** и верстать всё кодом, подготовьте проект.

1. Удалите файл Main.storyboard из проекта.

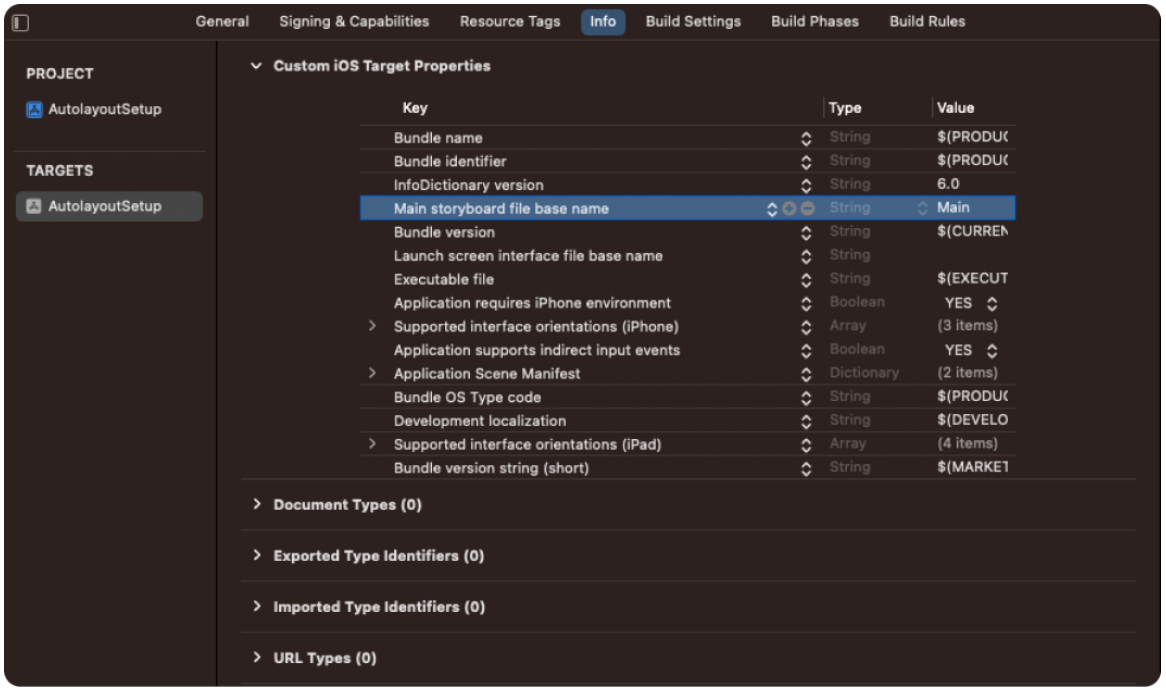


2. Удалите все ссылки и упоминания об использовании **Main.storyboard**.

   2.1 В файле Info.plist удалите строчку Storyboard Name.



   2.2 В файле проекта выберите в разделе Targets вкладку Info и удалить все вхождения Storyboard Name.



3. Обновите метод **scene(scene:willConnectTo:connectionOptions:)** в файле **SceneDelegate.swift.**

```
guard let windowScene = (scene as? UIWindowScene) else { return }
window = UIWindow(windowScene: windowScene)
window.rootViewController = ViewController()
window.makeKeyAndVisible()
```

В этом фрагменте кода мы вручную создаём **UIWindow** и задаём ему основной контроллер, с которого запускается приложение.

4. Обновите файл **AppDelegate.swift.**

   4.1 Добавьте ссылку на window.

```
var window: UIWindow?
```

   4.2 Создайте вручную **UIWindow** и инициализируйте стартовый экран, как в **SceneDelegate.**

```
window = UIWindow()
window?.rootViewController = ViewController()
window?.makeKeyAndVisible()
```

Проект подготовлен! Теперь всё в нём можно верстать кодом.

Создание элементов интерфейса

В вёрстке кодом вы не только вручную располагаете элементы на экране с помощью AutoLayout — вы создаёте элементы интерфейса (наследники UIView).

```
let label = UILabel()
label.translatesAutoresizingMaskIntoConstraints = false
label.text = "Hello, AutoLayout!"
label.font = .systemFont(ofSize: 20)
label.sizeToFit()
label.textAlignment = .center
```

При вёрстке кодом в каждой View уведите свойство **translatesAutoresizingMaskIntoConstraints** в **false.** Так вы укажете системе: вы используете **AutoLayout** для управления расположением и размерами элементов интерфейса. Если свойство оставить в значении **true** — система использует старую систему автоматического определения размеров и позиционирования элементов, а она с **AutoLayout** несовместима.

После вы сможете настраивать констрейнты AutoLayout для **label**.

```
label.centerXAnchor.constraint(equalTo: view.centerXAnchor),
label.centerYAnchor.constraint(equalTo: view.centerYAnchor),
```

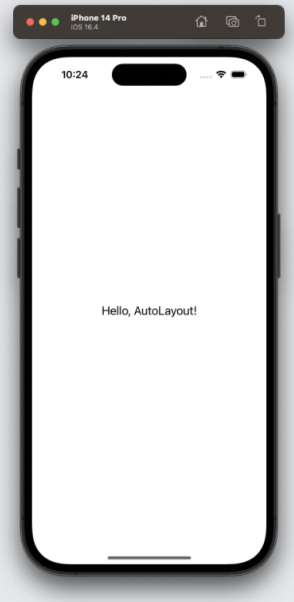
Код в примере привяжет центр по оси X и Y созданного **Label** к **View**. Но просто объявить констрейнты недостаточно — их нужно активировать. Для этого есть два способа:

1. У каждого отдельного констрейнта выставить свойство **isActive** в значение **true**.

   ```
   label.centerXAnchor.constraint(equalTo: view.centerXAnchor) = true
   label.centerYAnchor.constraint(equalTo: view.centerYAnchor) = true
   ```

2. Передать констрейнты массивом в метод класса **activate** у **NSLayoutConstraint**.

   ```
   NSLayoutConstraint.activate([
     label.centerXAnchor.constraint(equalTo: view.centerXAnchor),
     label.centerYAnchor.constraint(equalTo: view.centerYAnchor)
   ])
   ```



Anchors

- `widthAnchor` — ширина элемента.
- `heightAnchor` — высота элемента.
- `topAnchor` — верхний край элемента.
- `bottomAnchor` — нижний край элемента.
- `leadingAnchor` — левый край элемента (для леворукого языка интерфейса).
- `trailingAnchor` — правый край элемента (для леворукого языка интерфейса).
- `leftAnchor` — левый край элемента (для праворукого языка интерфейса).
- `rightAnchor` — правый край элемента (для праворукого языка интерфейса).
- `centerXAnchor` — горизонтальное центрирование элемента.
- `centerYAnchor` — вертикальное центрирование элемента.

Приоритеты

Приоритеты в **AutoLayout** позволяют указать, как важно следовать определённым констрейнтам. Они определяют, как система **AutoLayout** разрешит конфликты между констрейнтами.

Приоритеты можно установить для каждого констрейнта с помощью свойства **priority** типа **UILayoutPriority**. Значение приоритета может быть любым числом в диапазоне от **0** до **1000**, где **1000** — наивысший приоритет.

У **UILayoutPriority** есть готовые статичные свойства для самых частых приоритетов:

- `UILayoutPriorityRequired` эквивалентно значению **1000.**
- `UILayoutPriorityDefaultHigh` — **750.**
- `UILayoutPriorityDefaultLow` — **250.**
- `UILayoutPriorityFittingSizeLevel` — **50.**

```
let labelRightAnchor = label.rightAnchor.constraint(equalTo: view.rightAnchor)
labelRightAnchor.priority = .required
```

Если возникает конфликт между констрейнтами, система **AutoLayout** будет пытаться разрешить констрейнт с более высоким приоритетом. Если это невозможно — с более низким, или временно отключить некоторые констрейнты.

Назначать правильные приоритеты констрейнтам в **AutoLayout** — полезно, когда нужно управлять поведением интерфейса в зависимости от разных условий или размеров экрана.

Hugging и Compression Resistance

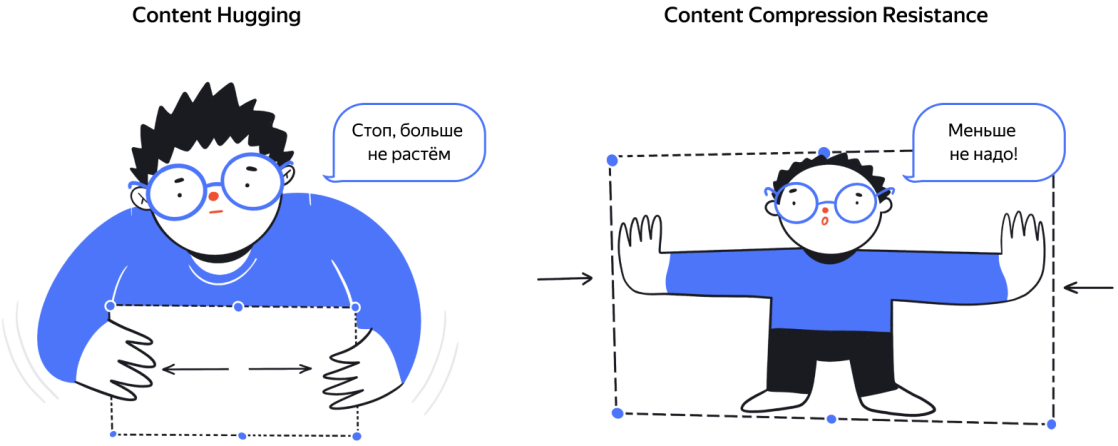
ContentHuggingPriority и **ContentCompressionResistancePriority** — два дополнительных приоритета, их можно устанавливать для ограничений **AutoLayout**.

ContentHuggingPriority определяет, нужно ли View сохранять размер. При высоком приоритете View будет стремиться сохранить минимальный размер, даже если это растянет другие View. При низком — View растянется, чтобы заполнить доступное пространство.

ContentCompressionResistancePriority указывает, сохранит View текущую размерность. При высоком приоритете View будет стремиться сохранять текущий размер, даже если это сожмёт другие View. При низком — View сожмётся, чтобы поместиться в доступное пространство.

Когда вы хотите контролировать, какие View будут растягиваться или сжиматься — используйте эти приоритеты. Если у вас две View рядом, и вы хотите, чтобы одна из них всегда была шире — назначьте ей более высокий **ContentHuggingPriority**.

```
label.setContentHuggingPriority(.required, for: .vertical)
label.setContentCompressionResistancePriority(.required, for: .vertical
```



Разница между Content Hugging и Content Compression Resistance

У каждой **UIView** и её наследников есть стандартные значения Hugging и Compression Resistance. Вот таблица для стандартных компонентов `UIDatePicker`.

Объект	Приоритет hugging (горизонтальный, вертикальный)	Приоритет compression resistance (горизонтальный, вертикальный)
`UIActivityIndicatorView`	`750,750`	`750,750`
`UIButton`	`250,250`	`750,750`
`UIDatePicker`	`750,750`	`750,750`
`UIImageView`	`250,250`	`750,750`
`UILabel`	`250,250`	`750,750`
`UIPageControl`	`750,750`	`750,750`
`UIPickerView`	`750,750`	`750,750`
`UIProgressView`	`250,750`	`750,750`
`UIScrollView`	`250,250`	`750,750`
`UISearchBar`	`250,750`	`750,750`
`UISegmentedControl`	`250,750`	`750,750`
`UISlider`	`250,750`	`750,750`
`UIStepper`	`750,750`	`750,750`

`UISwitch`	`750,750`	`750,750`
`UITabBar`	`250,750`	`750,750`
`UITextField`	`250,250`	`750,750`
`UITextView`	`250,250`	`750,750`
`UIToolbar`	`250,750`	`750,750`
`UIView`	`250,250`	`750,750`

Intrinsic Content Size

Intrinsic content size (внутренний размер содержимого) — размер, который **View** определяет как самый подходящий для отображения содержимого. Это может быть ширина и высота текста в UILabel или размер изображения в UIImageView.

Когда у View есть **Intrinsic content size**, она может автоматически рассчитывать свой размер — без необходимости задавать его через ограничения AutoLayout. Например, если у UILabel есть текст, он автоматически рассчитает свою ширину и высоту на основе этого текста.

Ограничения AutoLayout учитывают **Intrinsic content size** при расчёте размеров и позиций представлений. Эта информация может использовать, чтобы определять оптимальные размеры и размещение представлений на экране.

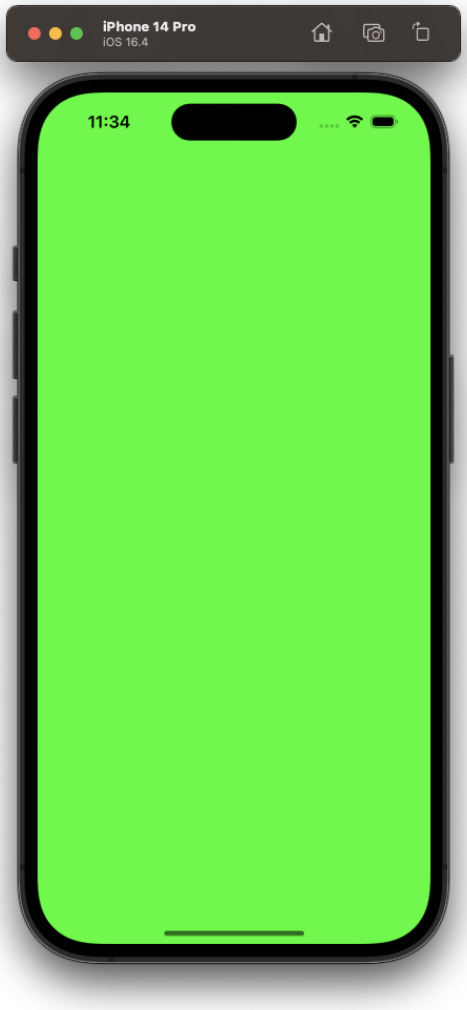
Если вы хотите контролировать размер представления — установите ограничения для его ширины и высоты. У ограничений будет приоритет над **Intrinsic content size**.

Intrinsic content size особенно полезен, когда вы хотите создать адаптивный интерфейс, который автоматически подстраивается под разные размеры содержимого. Например, если у вас есть UILabel с переменной длиной текста, он автоматически изменит ширину и высоту, чтобы вместить весь текст без обрезания или переноса строк.

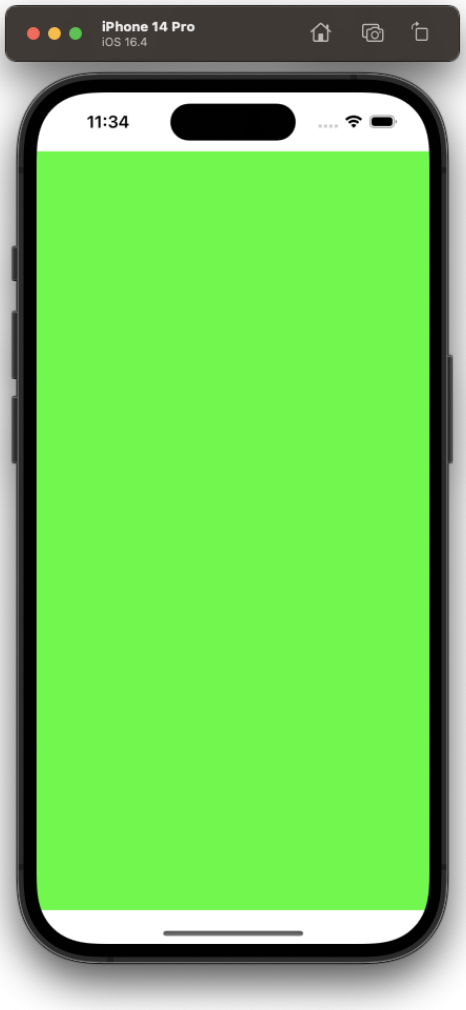
Safe area

Safe Area — область экрана, которая не перекрывается системными элементами интерфейса: Status Bar или Home Indicator. Учитывайте Safe Area при создании констрейнтов, чтобы интерфейс выглядел корректно на всех устройствах.

```
let guide = view.safeAreaLayoutGuide
NSLayoutConstraint.activate([
    containerView.topAnchor.constraint(equalTo: guide.topAnchor),
    containerView.leadingAnchor.constraint(equalTo: guide.leadingAnchor),
    containerView.trailingAnchor.constraint(equalTo: guide.trailingAnchor),
    containerView.bottomAnchor.constraint(equalTo: guide.bottomAnchor)
])
```



Без использования Safe Area



С использованием Safe Area

Полезные расширения

Добавление нескольких View на родительскую

```
extension UIView {
    func addSubviews(_ subviews: [UIView]) {
        subviews.forEach { addSubview($0) }
    }
}

let label = UILabel()
let button = UIButton()

view.addSubviews([label, button])
```

Привязывание границ к родительской View

```
extension UIView {
    @discardableResult func edgesToSuperview() -> Self {
        guard let superview = superview else {
            fatalError("View не в иерархии!")
        }
        translatesAutoresizingMaskIntoConstraints = false
        NSLayoutConstraint.activate([
            topAnchor.constraint(equalTo: superview.topAnchor),
            leftAnchor.constraint(equalTo: superview.leftAnchor),
            rightAnchor.constraint(equalTo: superview.rightAnchor),
            bottomAnchor.constraint(equalTo: superview.bottomAnchor)
        ])
        return self
    }
}

let containerView = UIView()
let imageView = UIImageView()

containerView.addSubview(imageView)
imageView.edgesToSuperview()
```

Яндекс Практикум